CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G6
System Integration and Architecture

System Design Document (SDD)

Project Title: StockEase

Prepared By: Batucan, Zly Hanson A.

Version: 1

Date: 2/21/2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2/21/2026	Batucan , Zly Hanson A.	Initial draft	Draft
1	2/28/2026	Batucan , Zly Hanson A.	Finalization	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 8
4.0 SYSTEM ARCHITECTURE 9
5.0 API CONTRACT & COMMUNICATION 10
Authentication Endpoints 11
User Registration 11
User Login 11
6.0 DATABASE DESIGN 13
7.0 UI/UX DESIGN 14
8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

StockEase is a stock and sales management system designed for small retail shops. It provides authentication, inventory tracking, sales recording, and automated stock deduction. The system includes a Spring Boot backend, React web application, and optional mobile application.

1.2 Objectives

- Implement secure authentication using JWT and BCrypt
- Provide full CRUD for products
- Record sales with automatic stock update
- Enforce role-based access control
- Integrate required external services
- Deploy complete system for final defense

1.3 Scope

Included Features:

- User registration and login
- Role-based access
- Product inventory management
- Sales recording
- Stock deduction automation
- Email notifications
- OAuth login
- File upload
- External API integration

Excluded Features:

- Advanced analytics
- AI forecasting
- Multi-branch management

1.0 INTRODUCTION

1.1 Purpose

This document provides the complete design plan for StockEase. It explains the system architecture, database structure, API design, and implementation approach. It describes how the backend layers are organized, how data is stored and related, and how endpoints are structured using REST principles. It also outlines security design, role-based access, and integration features required for the project. This document guides development, testing, and deployment to ensure the system is built according to the planned design.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: StockEase
Domain: E-commerce/Retail
Primary Users:

1. Small Business Owners

Problem Statement: Small shops often track inventory manually. This causes stock errors, lost sales data, and poor monitoring.
Solution: A centralized web-based system that records products and sales in real time.

2.2 Core User Journeys

Journey 1: Owner Registers and Sets Up Inventory

1. User visits the web application
2. Clicks Register and creates an account
3. Receives welcome email
4. Logs in using credentials
5. Navigates to Inventory Dashboard
6. Clicks Add Product
7. Enters product name, price, and quantity
8. Uploads product image
9. Saves product
10. Product appears in inventory list

Journey 2: Owner/Staff Records a Sale

1. Owner logs in
2. Navigates to Sales page
3. Selects product from list
4. Enters quantity
5. System validates available stock
6. Confirms sale
7. System deducts stock automatically
8. Sale record is saved

9. Dashboard updates stock count

Journey 3: Owner/Staff Monitors Inventory

1. Owner logs in
2. Opens Dashboard
3. Views total products and low stock alerts
4. Filters products by category
5. Updates product quantity if restocking
6. Saves changes
7. System updates inventory in database

Journey 4: STAFF Attempts Restricted Action

1. STAFF logs in
2. Navigates to inventory page
3. Delete button is not visible
4. If endpoint is accessed directly
5. System returns 403 Forbidden

2.3 Feature List MoSCoW

MUST HAVE

1. User authentication with JWT and BCrypt
2. Role based access control
3. Product CRUD operations
4. Sales recording with automatic stock deduction
5. Inventory dashboard
6. File upload for product images
7. Email sending for account and receipt
8. External API integration
9. Minimum five database tables
10. Role based access through adding a product and deleting

SHOULD HAVE

1. Product category filtering
2. Low stock notification display
3. Order or sales history view
4. Input validation messages

COULD HAVE

1. Basic sales analytics summary

2. Real time stock updates using polling
3. Export sales report to PDF

WON'T HAVE

1. Advanced forecasting
2. Multi branch inventory support
3. Complex accounting integration
4. Multi language support

2.4 Detailed Feature Specifications

Feature: User Authentication

Screens: Registration, Login, Profile

Fields: Email, Password, Confirm Password, First name, Last name

Validation: Valid email format, Password minimum 8 characters, Unique email

API Endpoints: POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me

Security: JWT authentication, BCrypt password hashing, Protected routes, Role validation

Feature: Role Based Access Control

Roles: Owner, Staff

Owner Permissions: Manage staff, manage products, view all sales, Record sales, View inventory

Staff Permissions: Record sales, View inventory

API Restriction:

Role verified at controller level
403 returned for unauthorized access

Feature: Product Management

Screens: Inventory List, Add Product,Edit Product

Fields: Name, Description, Price, Quantity, Category, Image

API Endpoints: GET /products, GET /products/{id}, POST /products, PUT /products/{id}
, DELETE /products/{id}

Validation: Price greater than 0, Quantity greater than or equal to 0, Name required

Feature: Sales Recording

Screens: Sales Form, Sales History

Fields: Product, Quantity, Total amount

Process: Validate stock, Calculate total, Deduct stock, Save sale, Send receipt email

API Endpoints: POST /sales, GET /sales, GET /sales/{id}

Business Rule: Sale cannot proceed if stock is insufficient

Feature: External API Integration

Purpose: Fetch exchange rate data

Usage: Convert total sales into another currency on dashboard

Feature: File Upload

Upload product image
Store file on server
Save file path in database
Display image in product listing

Feature: Email Sending

Account related email
Welcome email after registration

System notification email
Receipt email after sale

2.5 Acceptance Criteria

- **AC 1 Successful Registration**
  Given I am a new user
  When I submit valid registration details
  Then my account is created
  And password is stored hashed
  And I receive welcome email
  And I receive JWT token
- **AC 2 Successful Sale Transaction**
  Given product stock is available
  When I record a sale
  Then stock decreases correctly
  And sale record is stored
  And receipt email is sent
  And updated stock appears on dashboard
- **AC 3 Stock Validation**
  Given product quantity is 5
  When I attempt to sell 6 items
  Then system rejects transaction
  And displays insufficient stock message
- **AC 4 Product Management**
  Given I am logged in as owner
  When I add or update product details
  Then product list reflects the changes immediately
- **AC 5 Role Restriction**
  Given I am logged in as staff
  When I attempt to delete a product
  Then system returns 403 Forbidden

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance

• API response time must be under 2 seconds for 95 percent of requests
• Database queries must complete within 500 milliseconds
• System must support at least 100 concurrent users
• Stock deduction must complete within the same transaction as sale creation
• File upload limited to 5MB per image

3.2 Security

• All communication must use HTTPS
• Authentication must use JWT access tokens
• Passwords must be hashed using BCrypt with 12 salt rounds

- System must enforce two roles, OWNER and STAFF
- Role validation must occur at API level
- Restricted UI elements must not render for STAFF
- Unauthorized access must return 403 Forbidden
- SQL injection must be prevented using JPA parameter binding
- DTOs must exclude password fields
- Rate limit set to 100 requests per minute per IP

3.3 Reliability

- Sale recording and stock deduction must use database transaction
- If stock validation fails, sale must not be saved
- System must rollback on processing errors
- Data must persist in PostgreSQL without duplication

3.4 Usability

- Dashboard must clearly show total products and low stock alerts
- Sale process must require fewer than 5 input steps
- Error messages must clearly state corrective action
- Layout must be responsive across desktop and mobile

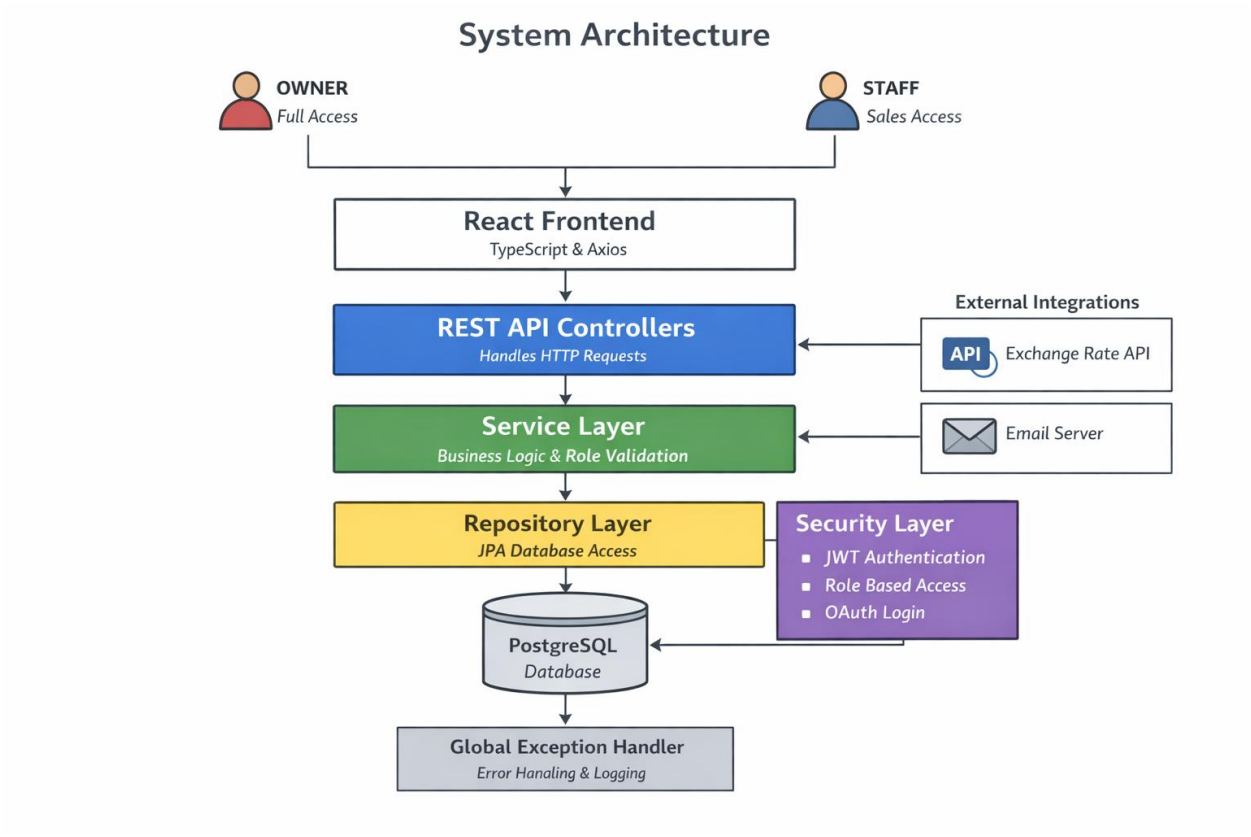
3.5 Maintainability

- Backend must follow layered architecture
- Controllers must not contain business logic
- Global exception handler must centralize error responses
- REST naming conventions must be consistent

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Roo
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	/api/v1
Format	JSON
Authentication	Authorization header with Bearer token
Response Structure	{

	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": "<email>",
	"password": "<password>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "OWNER or STAFF"
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	},
	"accessToken": "<token>",
	"refreshToken": "<token>"
	},
	"error": null,
	"timestamp": "<ISO8601 timestamp>"
	} "firstname": <firstname>,
	"lastname": <lastname>,

	},
	 “accessToken”: <token>,
	 “refreshToken”: <token>,
	 },
	 "error": {
	 "code": string,
	 "message": string,
	 "details": object|null
	 },
	 "timestamp": string
	 }

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": "<email>",
	"password": "<password>"
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	},
	"accessToken": "<token>",
	"refreshToken": "<token>"
	},
	"error": null,
	"timestamp": "<ISO8601 timestamp>"
	}

Get Current User

Description	Get authenticated user information
API URL	/auth/me

HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token Required
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": {
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	},
	"error": null,
	"timestamp": "<ISO8601 timestamp>"
	}

5.3 Product Endpoints

Create Product

Description	Create new product
API URL	/products
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token Required Role Required: OWNER

Request Payload	{
	"name": "<product name>",
	"description": "<description>",
	"price": 0.00,
	"quantity": 0
	}

Response Structure	{
	"success": boolean,
	"data": {

	"id": 1, "name": "<product name>", "price": 0.00, "quantity": 0 }, "error": null, "timestamp": "<ISO8601 timestamp>" }

Get All product

Description	Retrieve all products
API URL	/products
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token Required
Request Payload	None

Response Structure	{
	"success": boolean,
	"data": [
	{
	"id": 1,
	"name": "<product name>",
	"price": 0.00,
	"quantity": 0
	}
	],
	"error": null,
	"timestamp": "<ISO8601 timestamp>"
	}

Delete Product

Description	Get Current User
API URL	/auth/login

HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": "<email>",
	"password": "<password>"
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"email": "<email>",
	"firstname": "<firstname>",
	"lastname": "<lastname>",
	"role": "<role>"
	},
	"accessToken": "<token>",
	"refreshToken": "<token>"
	},
	"error": null,
	"timestamp": "<ISO8601 timestamp>"
	}

5.4 Sales Endpoint

Record Sale

Description	Record a sale and deduct stock
API URL	/sales
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token Required Roles Allowed: OWNER, STAFF
Request Payload	{ "items": [{ "productId": 1,

	"quantity": 2 }] }

Response Structure	{
	"success": boolean,
	"data": {
	"saleId": 10,
	"totalAmount": 500.00,
	"createdAt": "<timestamp>"
	},
	"error": null,
	"timestamp": "<ISO8601 timestamp>"
	}

5.5 Error Handling

HTTP Status Codes

- 200 OK
- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 500 Internal Server Error

Error Code Examples

Example 1	json { "success": false, "data": null,

	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 3	{
	"success": false,
	"data": null,
	"error": {
	"code": "<ERROR_CODE>",
	"message": "<Short description>",
	"details": "<Optional details>"
	},
	"timestamp": "<ISO8601 timestamp>"
	}

Common Error Codes

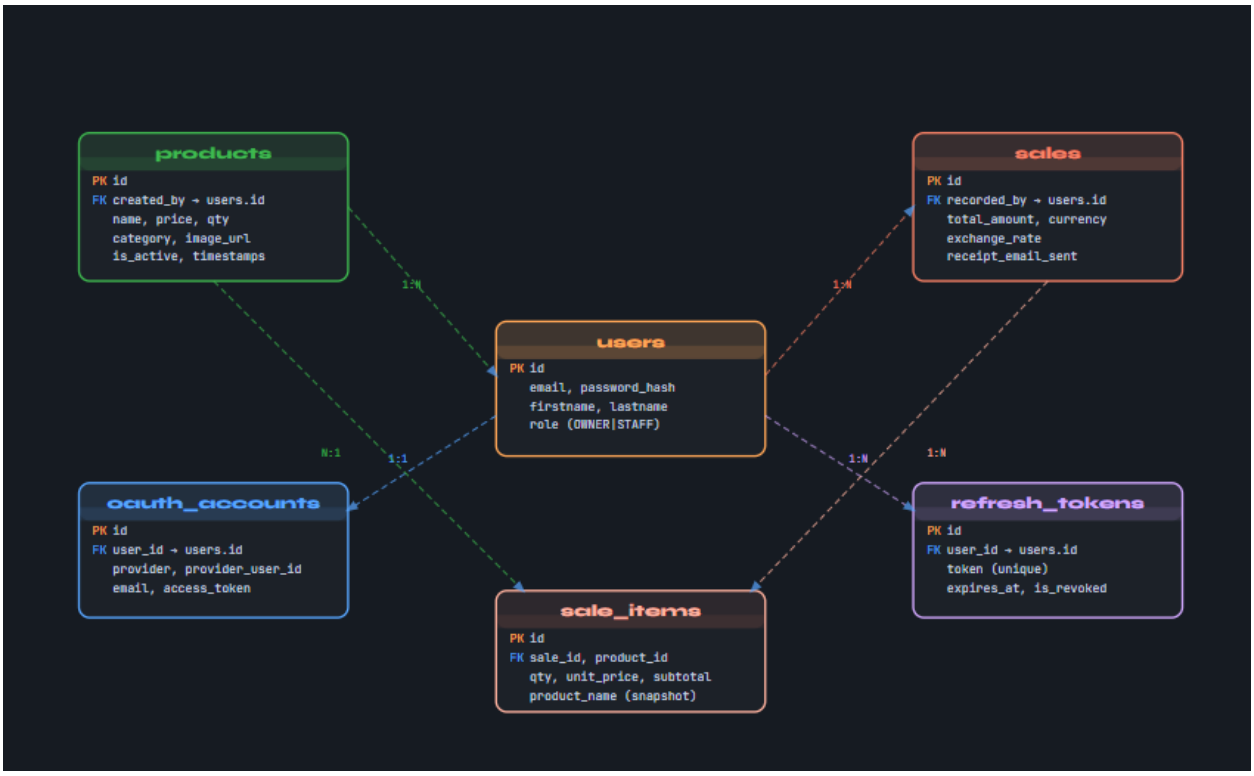
- AUTH-001 Invalid credentials

- AUTH-002 Token expired
- AUTH-003 Insufficient permissions
- VALID-001 Validation failed
- BUSINESS-001 Insufficient stock
- SYSTEM-001 Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships

ONE-TO-MANY

users → sales

FK: sales.recorded_by → users.id | One user can record many sales

ONE-TO-MANY

sales → sale_items

FK: sale_items.sale_id → sales.id | One sale has many line items
MANY-TO-ONE

sale_items → products
FK: sale_items.product_id → products.id | Many items reference one product
ONE-TO-MANY

users → refresh_tokens
FK: refresh_tokens.user_id → users.id | One user can have many tokens
ONE-TO-ONE

users → oauth_accounts
FK: oauth_accounts.user_id → users.id | One user linked to one OAuth account
ONE-TO-MANY

users → products
FK: products.created_by → users.id | One owner manages many products

Key Tables:

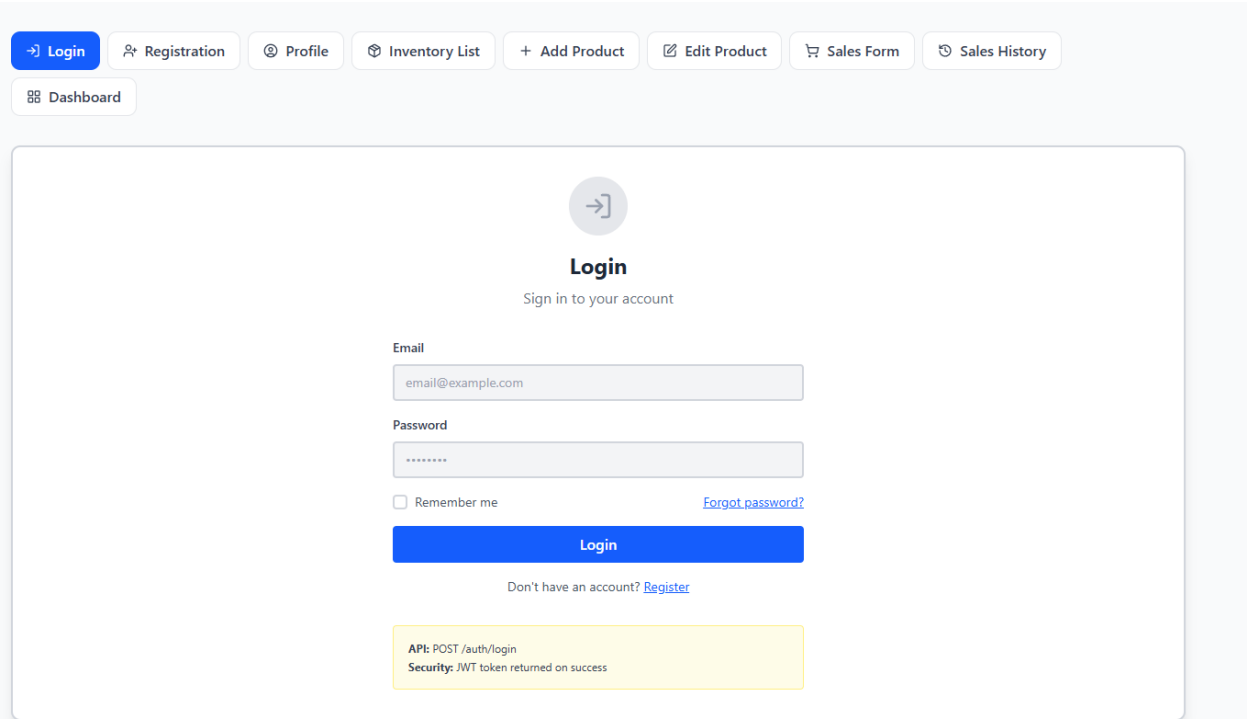
1. **Users** - email, BCrypt hash, role (OWNER/STAFF), provider flag
2. **oauth_accounts** - links provider_user_id back to your user record + generates your own JWT
3. **refresh_tokens** - stores tokens with expiry + revocation flag
4. **products** - full CRUD, image_url for file upload feature
5. **sales** - stores exchange_rate snapshot from External API
6. **sale_items** - snapshots product_name & unit_price at time of sale (good normalization practice)

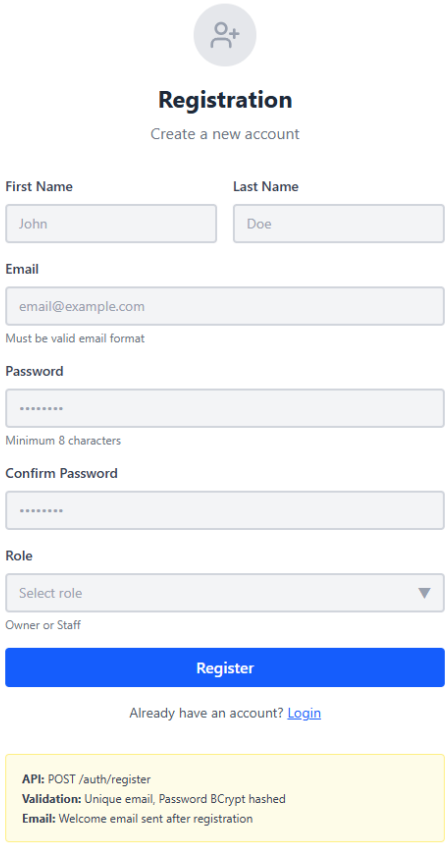
Table Structure Summary:

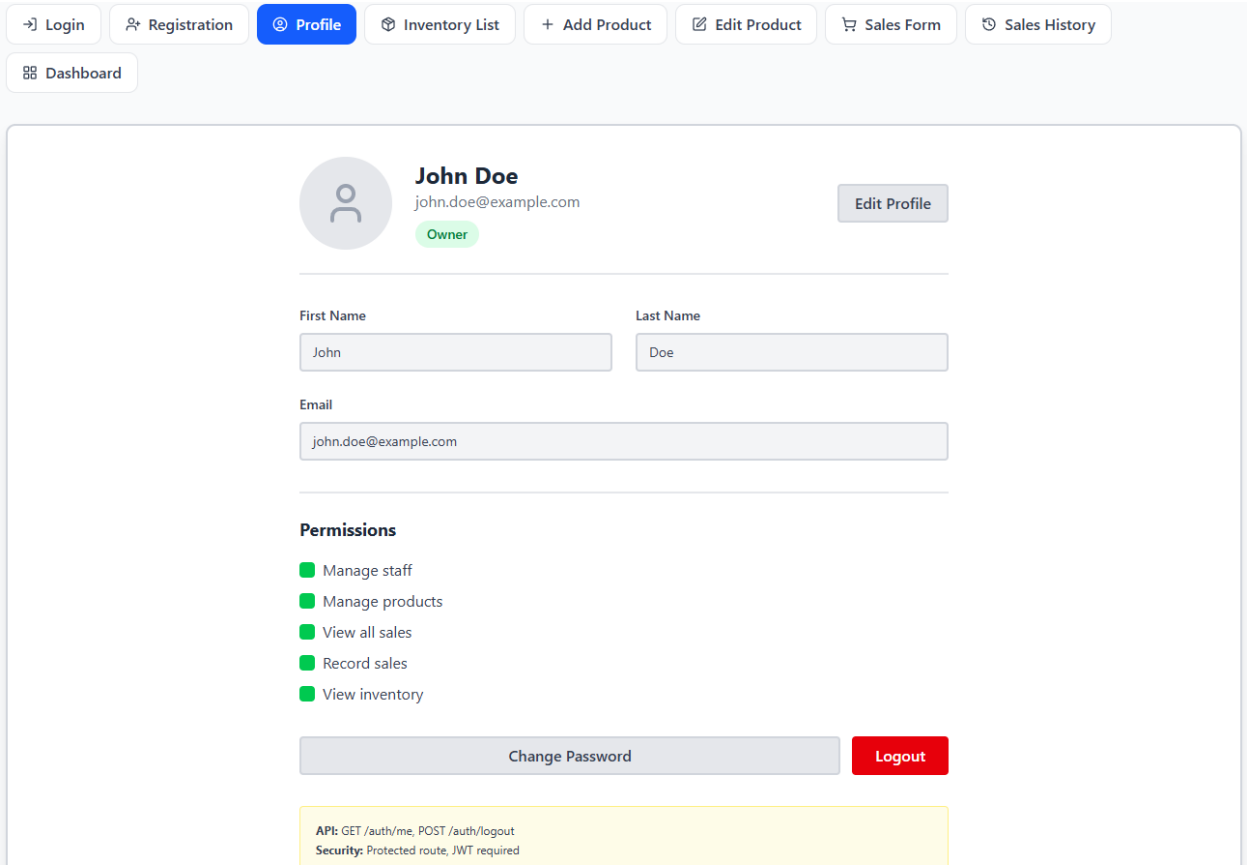
- **users:** id, email, password_hash, firstname, lastname, role, provider, is_verified, created_at, updated_at
- **oauth_accounts:** id, user_id, provider, provider_user_id, email, access token, created_at
- **refresh_tokens:** id, user_id, token, expires_at, is_revoked, created_at
- **products:** id, created_by, name, description, price, quantity, category, image_url, is_active, created_at, updated_at
- **sales:** id, recorded_by, total_amount, currency_code, exchange_rate, receipt_email_sent, created_at
- **sales_items:** id, sale_id, product_id, product_name, quantity, unit_price, subtotal

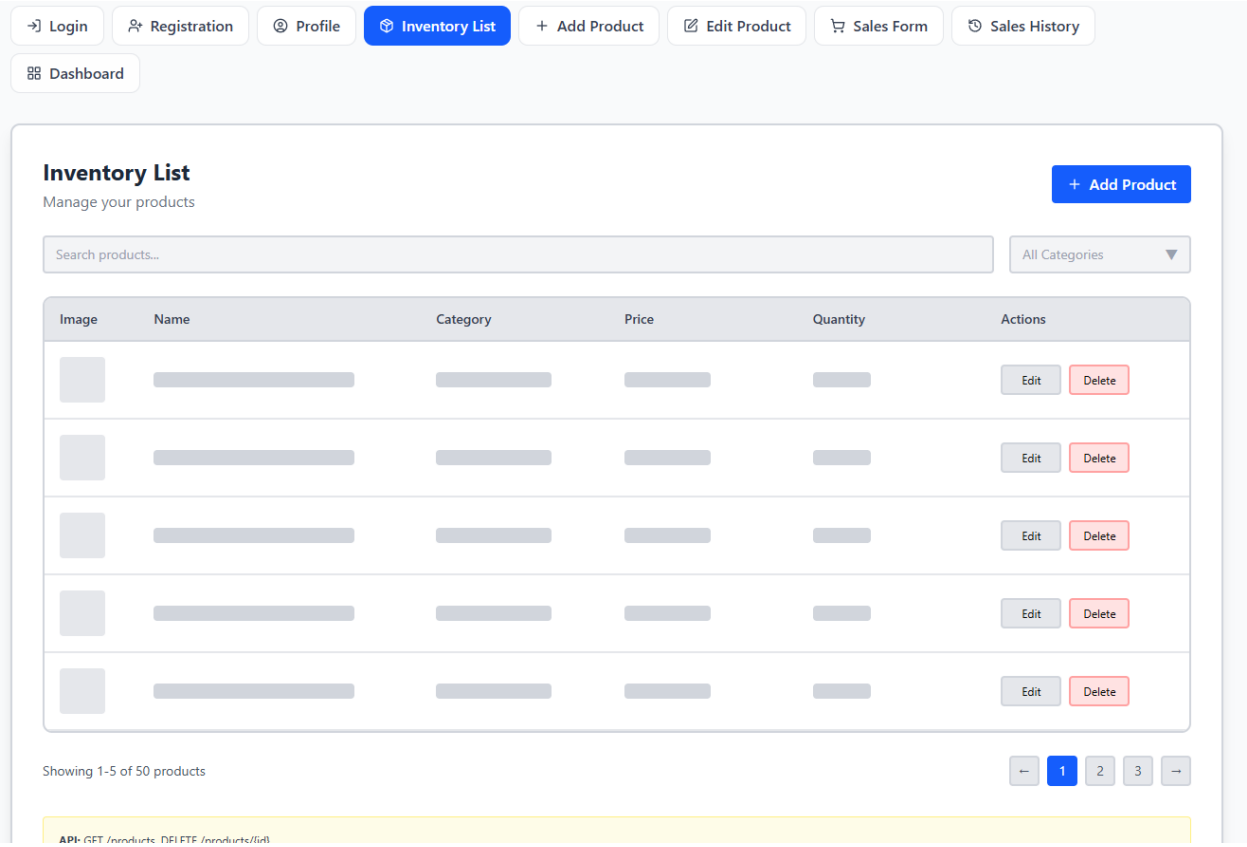
7.0 UI/UX DESIGN

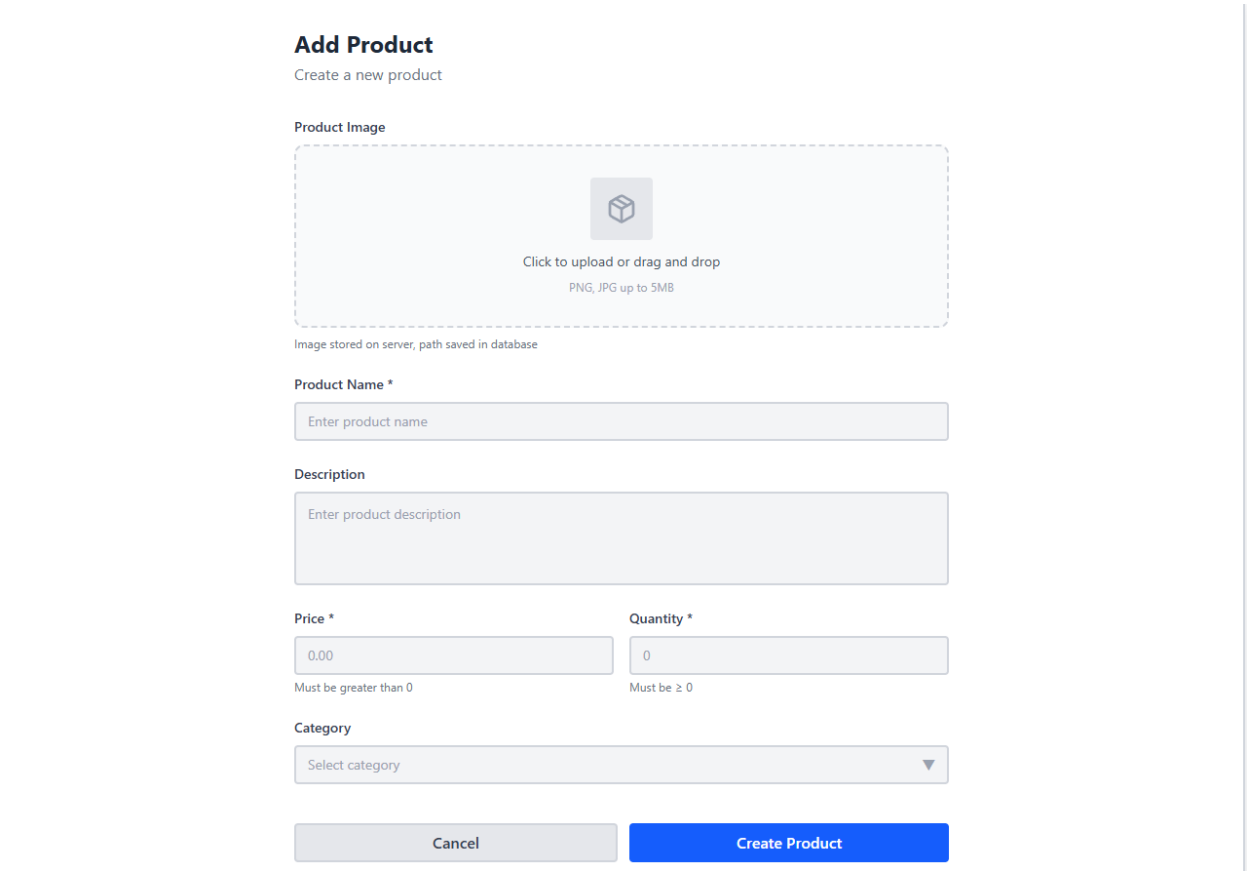
7.1 Web Application Wireframes




Registration
Create a new account
First Name
Last Name
John
Doe
Email
email@example.com
Must be valid email format
Password
Minimum 8 characters
Confirm Password
Role
Select role
Owner or Staff
Register
Already have an account? Login
API: POST /auth/register
Validation: Unique email, Password BCrypt hashed
Email: Welcome email sent after registration


Login
Registration
Profile
Inventory List
Add Product
Edit Product
Sales Form
Sales History
Dashboard
John Doe
john.doe@example.com
Owner
Edit Profile
First Name
John
Last Name
Doe
Email
john.doe@example.com
Permissions
Manage staff
Manage products
View all sales
Record sales
View inventory
Change Password
Logout
API: GET /auth/me, POST /auth/logout
Security: Protected route, JWT required


Login
Registration
Profile
Inventory List
Add Product
Edit Product
Sales Form
Sales History
Dashboard
Inventory List
Manage your products
Add Product
Search products...
All Categories
Image
Name
Category
Price
Quantity
Actions
Edit
Delete
Showing 1-5 of 50 products
1
2
3


Add Product
Create a new product
Product Image
Click to upload or drag and drop
PNG, JPG up to 5MB
Image stored on server, path saved in database
Product Name *
Enter product name
Description
Enter product description
Price *
0.00
Must be greater than 0
Quantity *
0
Must be ≥ 0
Category
Select category
Cancel
Create Product

Edit Product

Update product information

Product Image

Click to upload new image

PNG, JPG up to 5MB

Product Name *

Wireless Mouse

Description

High-quality wireless mouse with ergonomic design

Price *

29.99

Quantity *

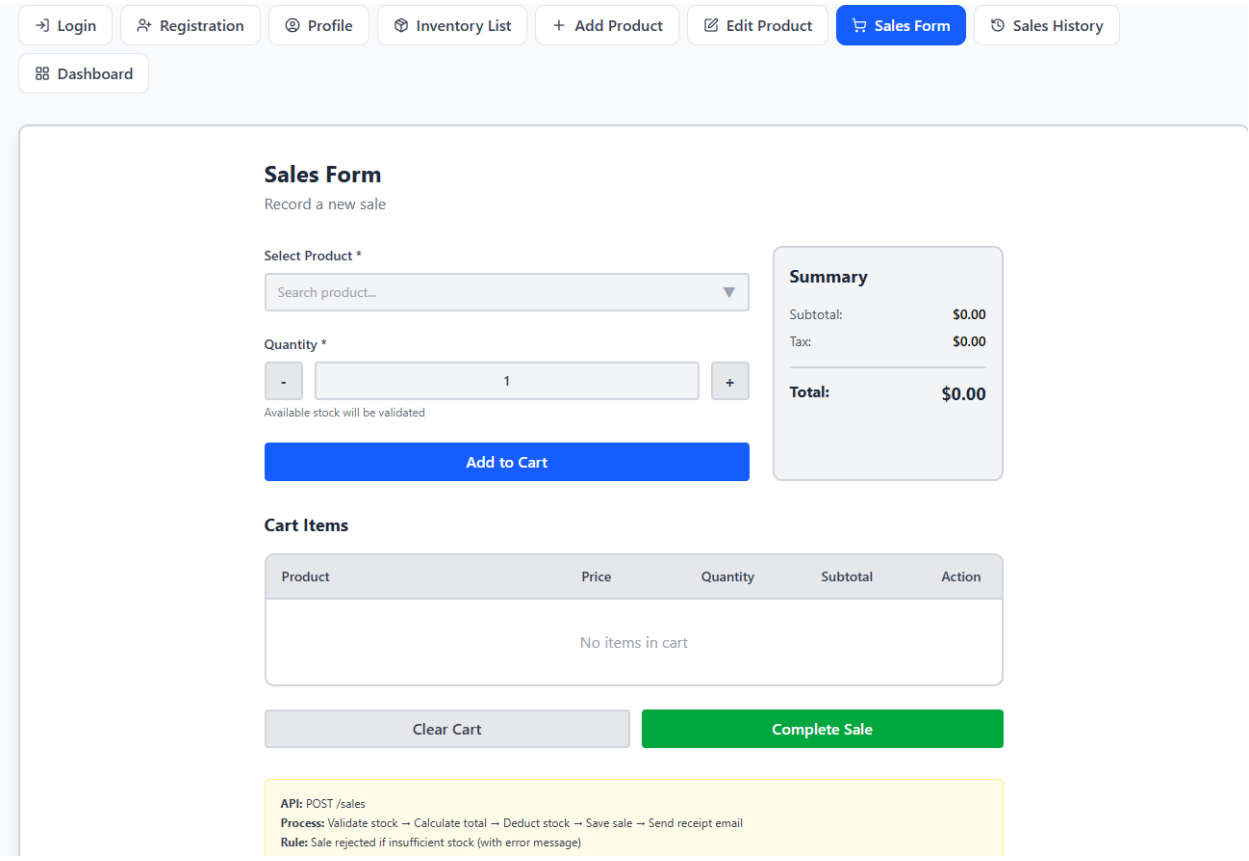
45

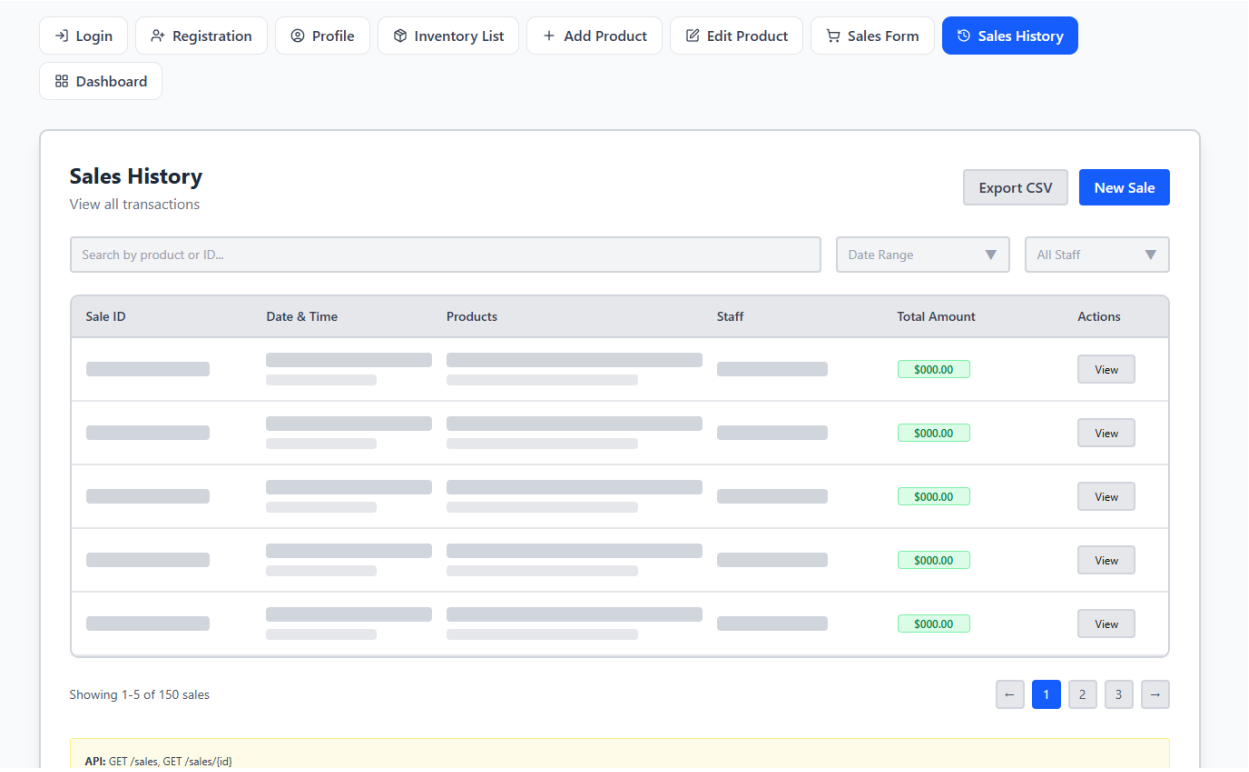
Category

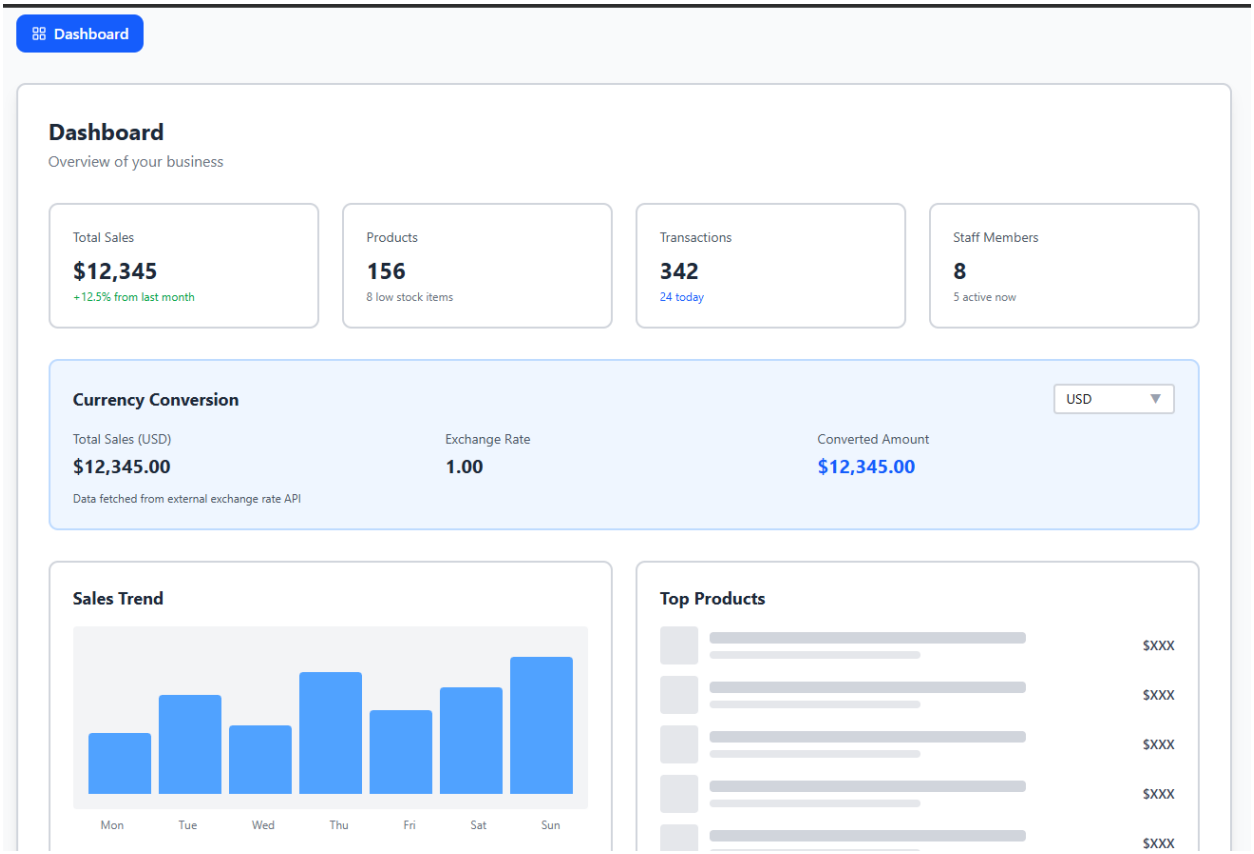
Electronics

Cancel | Update Product

API: GET /products/{id}, PUT /products/{id}
Permission: Owner only

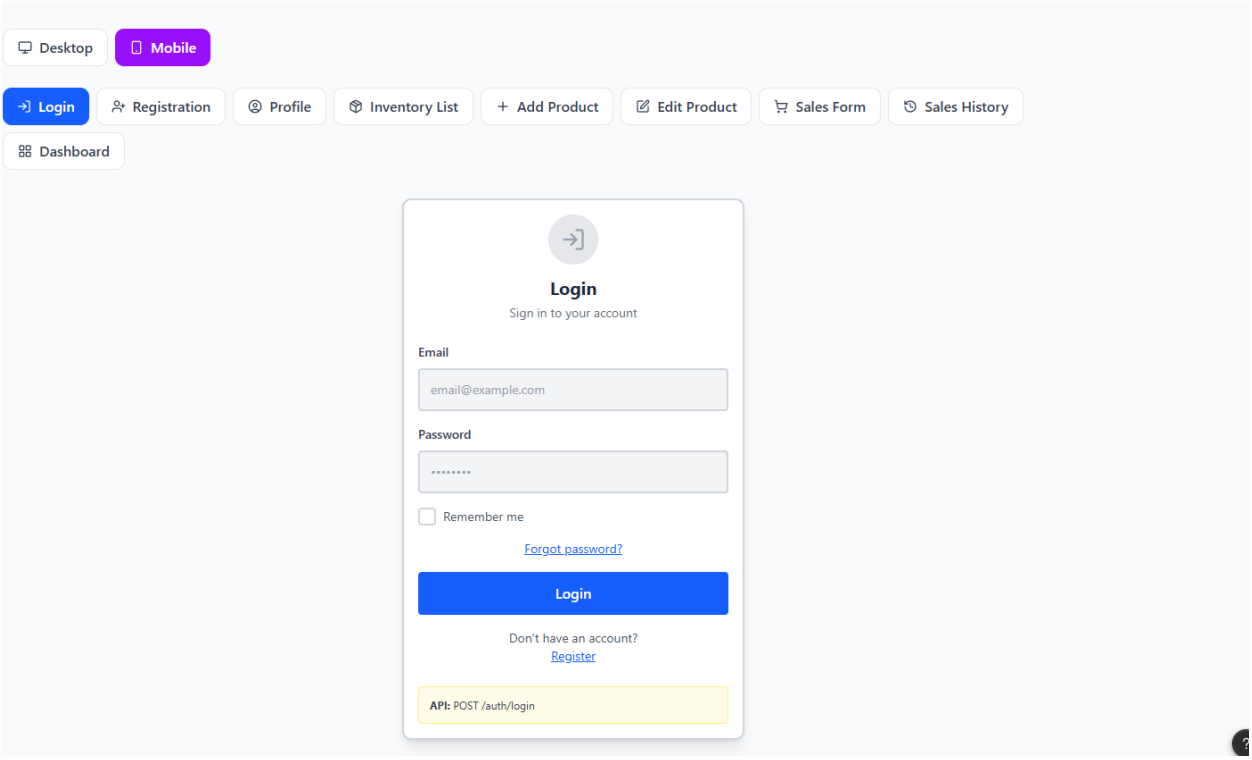

Login
Registration
Profile
Inventory List
Add Product
Edit Product
Sales Form
Sales History
Dashboard
Sales Form
Record a new sale
Select Product *
Search product...
Quantity *
-
1
+
Available stock will be validated
Add to Cart
Summary
Subtotal: $0.00
Tax: $0.00
Total: $0.00
Cart Items
Product
Price
Quantity
Subtotal
Action
No items in cart
Clear Cart
Complete Sale
API: POST /sales
Process: Validate stock → Calculate total → Deduct stock → Save sale → Send receipt email
Rule: Sale rejected if insufficient stock (with error message)

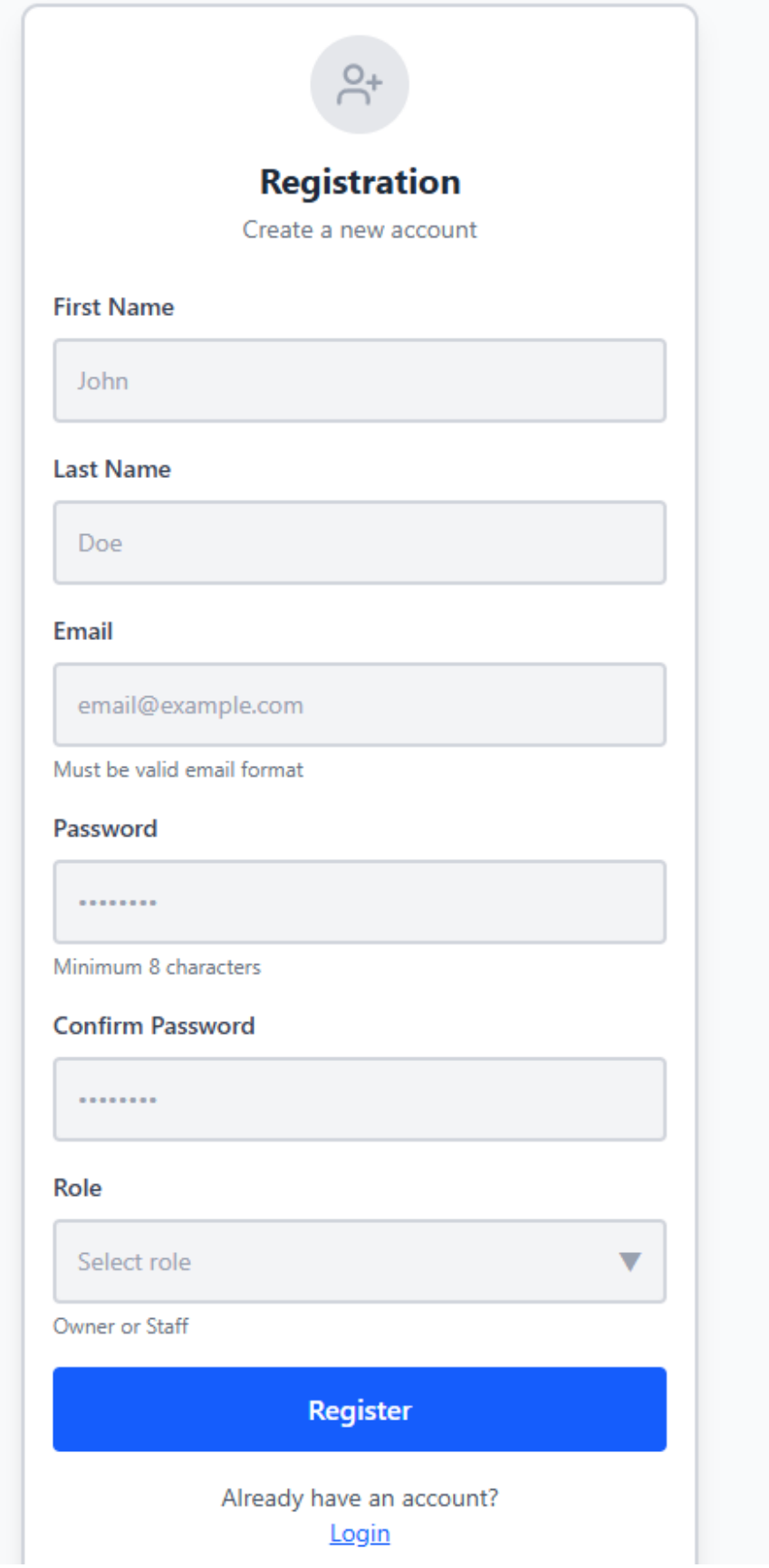

Login
Registration
Profile
Inventory List
Add Product
Edit Product
Sales Form
Sales History
Dashboard
Sales History
View all transactions
Export CSV
New Sale
Search by product or ID...
Date Range
All Staff
Sale ID
Date & Time
Products
Staff
Total Amount
Actions
$000.00
View
Showing 1-5 of 150 sales
1
2
3
API: GET /sales, GET /sales/{id}

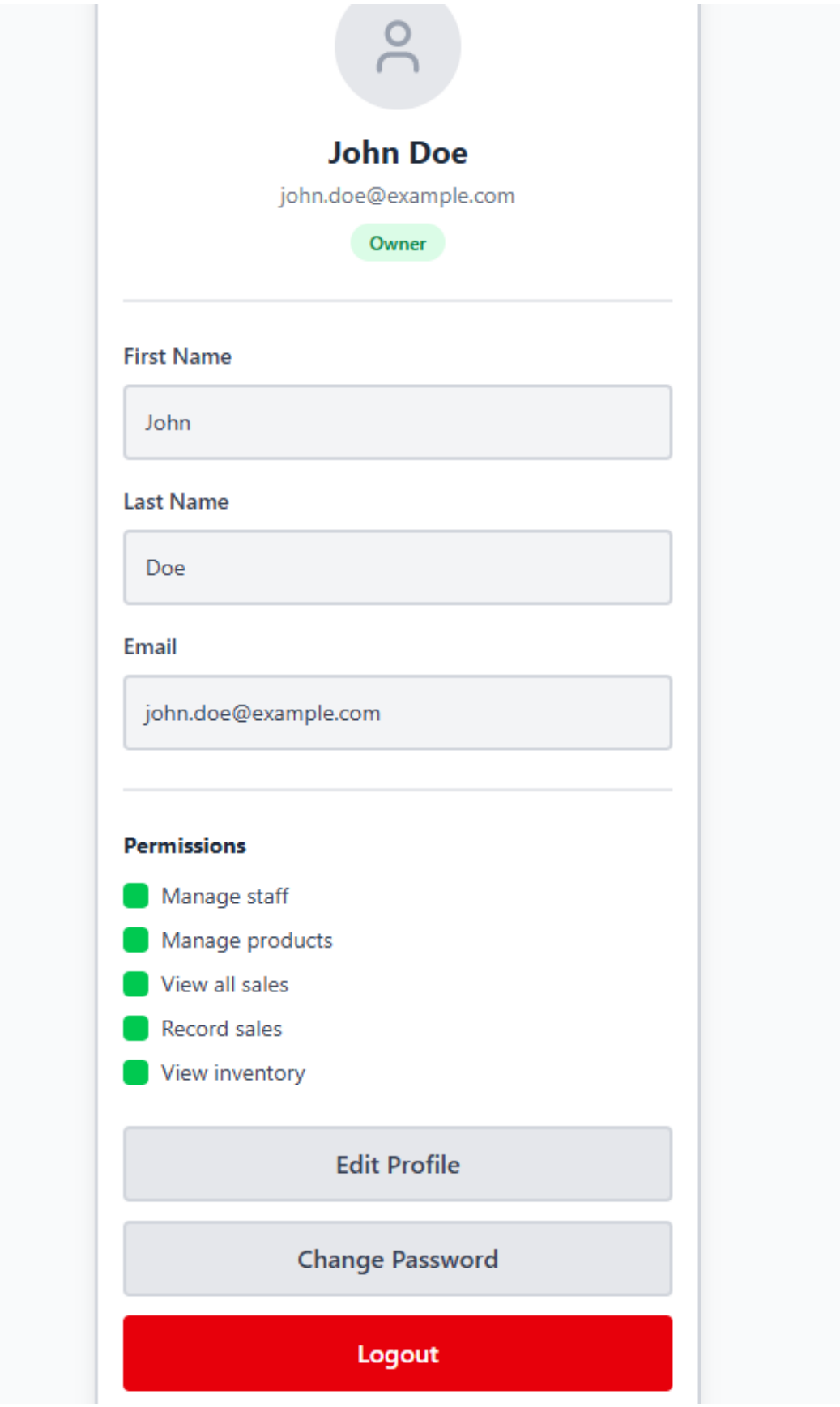


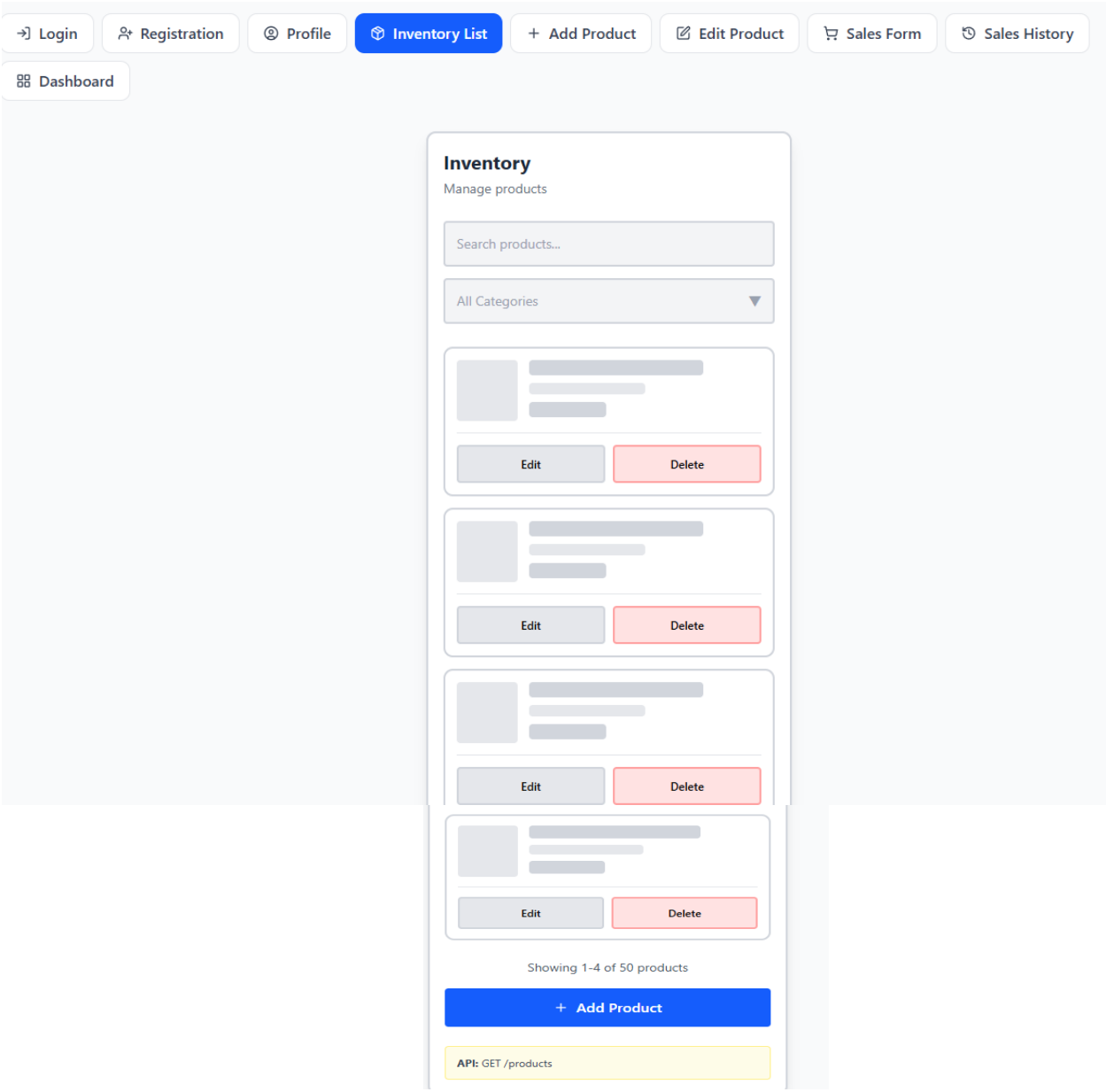
7.2 Mobile Application Wireframes

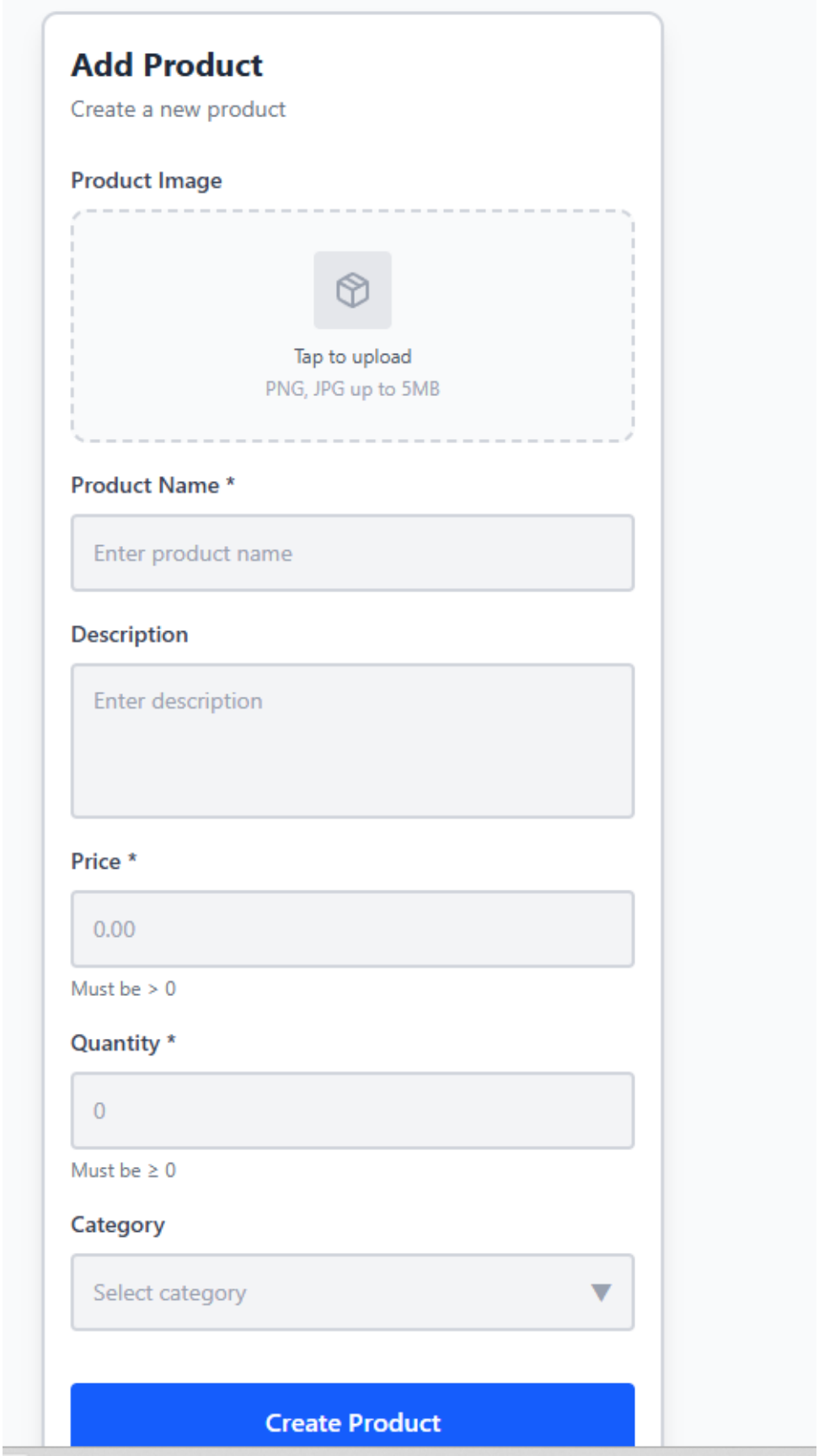
Note: This should be wireframes from Figma

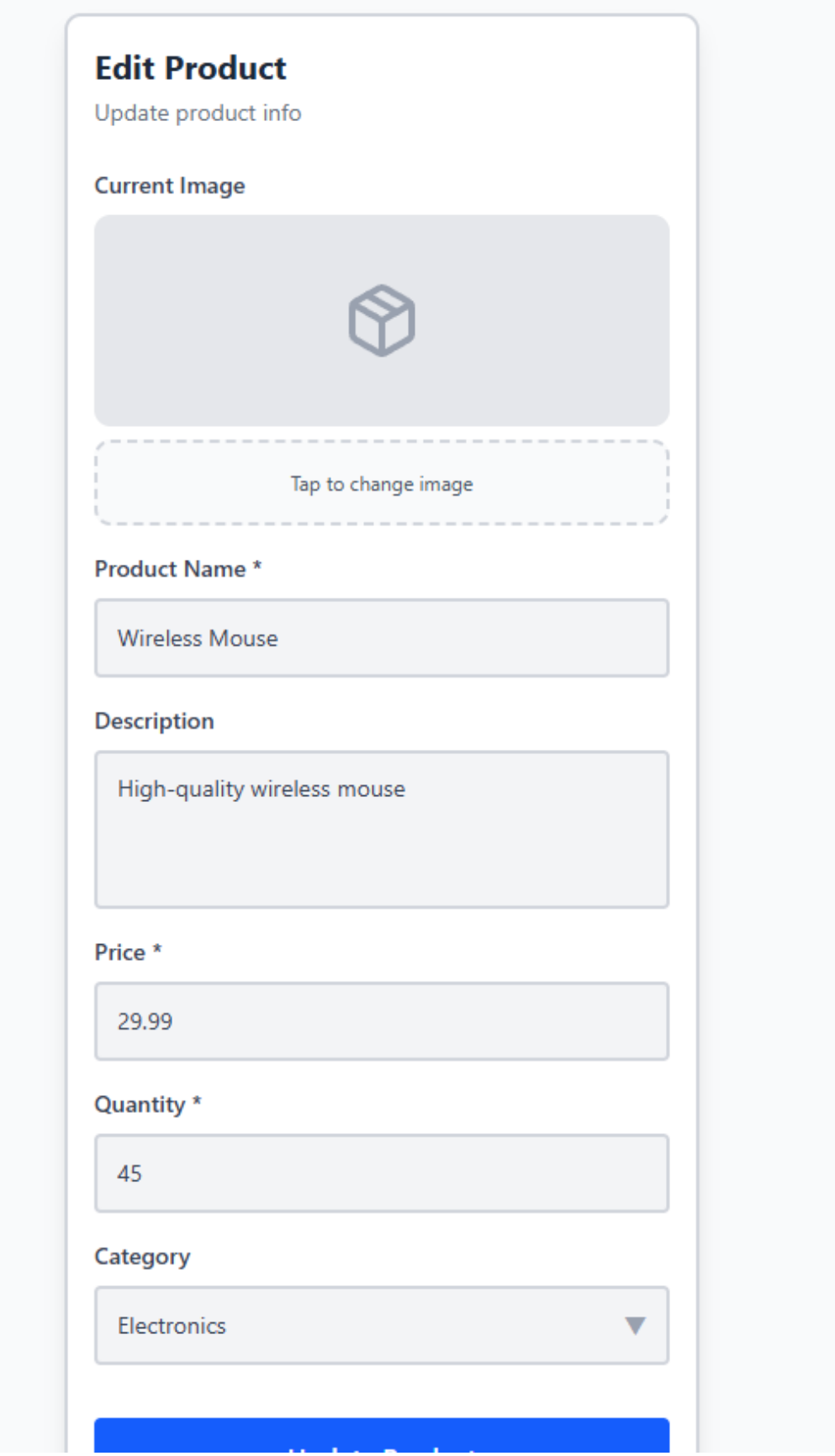

Desktop
Mobile
Login
Registration
Profile
Inventory List
Add Product
Edit Product
Sales Form
Sales History
Dashboard
Login
Sign in to your account
Email
email@example.com
Password
Remember me
Forgot password?
Login
Don't have an account?
Register
API: POST /auth/login


Registration
Create a new account
First Name
John
Last Name
Doe
Email
email@example.com
Must be valid email format
Password
Minimum 8 characters
Confirm Password
Role
Select role
Owner or Staff
Register
Already have an account?
Login


John Doe
john.doe@example.com
Owner
First Name
John
Last Name
Doe
Email
john.doe@example.com
Permissions
Manage staff
Manage products
View all sales
Record sales
View inventory
Edit Profile
Change Password
Logout


Login
Registration
Profile
Inventory List
Add Product
Edit Product
Sales Form
Sales History
Dashboard
Inventory
Manage products
Search products...
All Categories
Edit
Delete
Edit
Delete
Edit
Delete
Edit
Delete
Showing 1-4 of 50 products
Add Product
API: GET /products


Add Product
Create a new product
Product Image
Tap to upload
PNG, JPG up to 5MB
Product Name *
Enter product name
Description
Enter description
Price *
0.00
Must be > 0
Quantity *
0
Must be ≥ 0
Category
Select category
Create Product


Edit Product
Update product info
Current Image
Tap to change image
Product Name *
Wireless Mouse
Description
High-quality wireless mouse
Price *
29.99
Quantity *
45
Category
Electronics

Sales Form

Record a new sale

Select Product *

Search product...

Quantity *

- 1 +

Add to Cart

Summary

Subtotal:	$0.00
Tax:	$0.00
Total:	**$0.00**

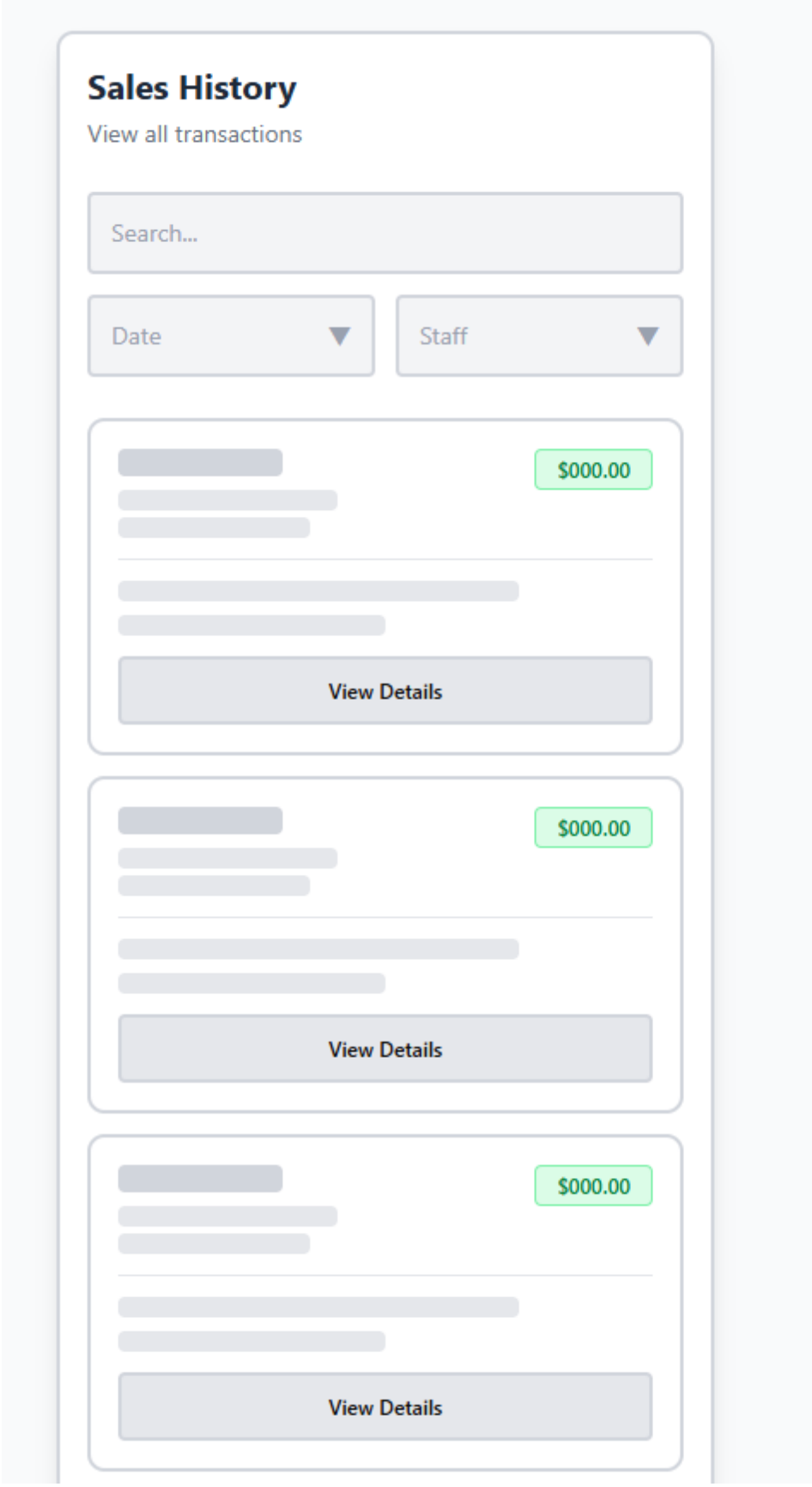
Cart Items

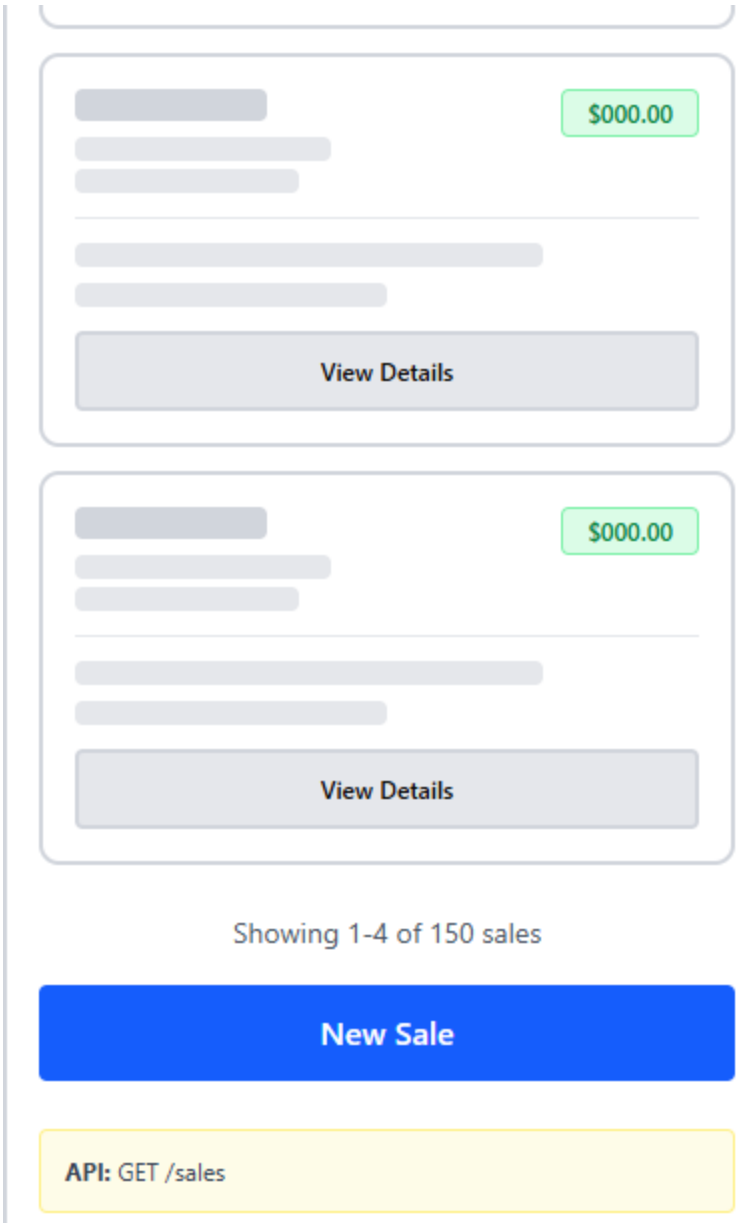
No items in cart

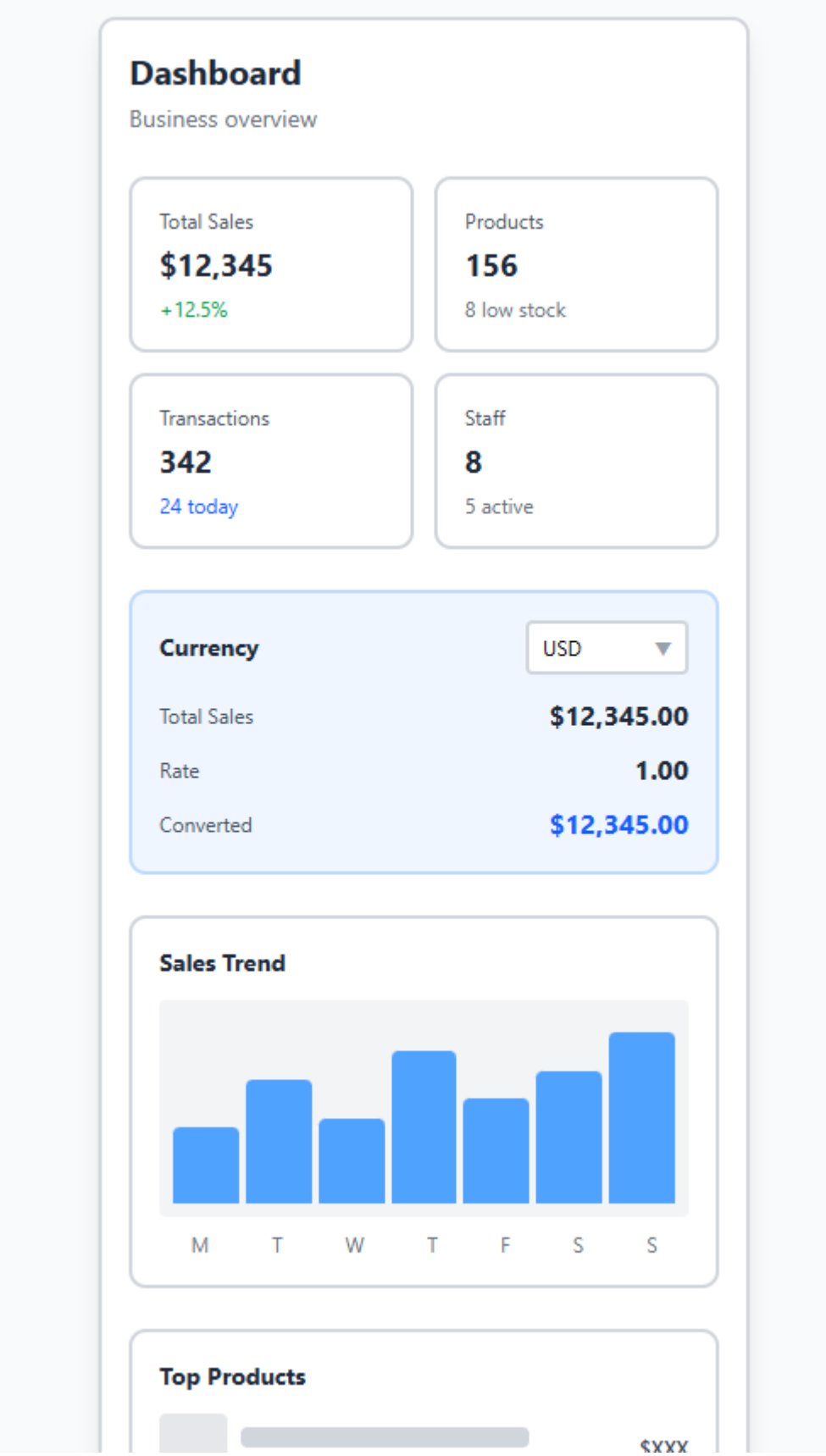
Complete Sale

Clear Cart

API: POST /sales
Process: Validate → Deduct stock → Send receipt


Sales History
View all transactions
Search...
Date
Staff
$000.00
View Details
$000.00
View Details
$000.00
View Details


$000.00
View Details
$000.00
View Details
Showing 1-4 of 150 sales
New Sale
API: GET /sales


Dashboard
Business overview
Total Sales
$12,345
+12.5%
Products
156
8 low stock
Transactions
342
24 today
Staff
8
5 active
Currency
USD
Total Sales
$12,345.00
Rate
1.00
Converted
$12,345.00
Sales Trend
M
T
W
T
F
S
S
Top Products
$XXX

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1 – 2)

This phase covers all design and documentation work before any code is written. The goal is to finalize the SDD, ERD, API contract, and wireframes so that development can proceed without ambiguity.

Week 1: Requirements & Architecture

Day	Task / Deliverable
Day 1 – 2	Project setup, repository structure (IT342_StockEase_Section_Lastname), initial documentation
Day 3 – 4	Complete Functional Requirements Specification (FRS) and Non-Functional Requirements (NFR)
Day 5 – 7	System architecture design — layered architecture diagram, technology stack decision

Week 2: Detailed Design

Day	Task / Deliverable

Day 1 – 2	API contract specification — all endpoints, request/response payloads, error codes
Day 3 – 4	Database design — ERD finalization, table structures, relationships, normalization review
Day 5 – 6	UI/UX wireframes for web (Inventory Dashboard, Sales Form, Login/Register)
Day 7	Mobile wireframes; implementation plan finalization; SDD review and sign-off

---

Phase 2: Backend Development (Week 3 – 4)

This phase builds the entire Spring Boot backend — authentication, core business logic, integrations, and all REST endpoints. This is the critical path of the project.

Week 3: Foundation & Authentication

Day	Task / Deliverable
Day 1	Spring Boot project setup — dependencies (Spring Security, JPA, JWT, Mail, OAuth2), PostgreSQL config
Day 2	Database entity mapping — User, Product, Sale, SaleItem, RefreshToken, OAuthAccount
Day 3	JWT authentication — register, login, /auth/me, BCrypt hashing, token generation
Day 4	Google OAuth 2.0 integration — OAuth callback, user linking, JWT generation post-OAuth
Day 5	Role-based access control — OWNER/STAFF enforcement at controller and service layers; 403 handling

Week 4: Core Features & Integrations

Day	Task / Deliverable
Day 1	Product CRUD — POST/GET/PUT/DELETE /products, OWNER-only restrictions, image_url field support
Day 2	File upload — product image upload endpoint, server storage, DB path linking, size validation (5MB)

Day 3	Sales recording — POST /sales, stock validation, transactional stock deduction, sale_items creation
Day 4	External API integration — exchange rate fetch, rate stored on sale record, dashboard display
Day 5	Email sending — welcome email (SMTP) on register; receipt email after successful sale; global exception handler; DTO audit

---

Phase 3: Web Application (Week 5 – 6)

The React + TypeScript web frontend is built here. All screens are connected to the live backend API.

Week 5: Frontend Foundation

Day	Task / Deliverable
Day 1	React 18 + TypeScript + Tailwind CSS project setup, Axios HTTP client configuration, routing setup
Day 2	Authentication pages — Login, Register, Google OAuth redirect handling, JWT storage
Day 3	Inventory Dashboard — product list, low stock alerts, total product count, category filter
Day 4	Add/Edit Product pages — form with image upload, validation messages, OWNER-only access
Day 5	Role-based UI — hide delete/edit controls for STAFF; protected route wrappers

Week 6: Complete Web Features

Day	Task / Deliverable
Day 1	Sales recording form — product selector, quantity input, stock validation feedback, submit
Day 2	Sales history view — list of past sales with totals and exchange rate display
Day 3	Dashboard enhancements — converted currency display using stored exchange rate data
Day 4	Responsive design polish — mobile-first breakpoints, Tailwind refinement across all pages

Day 5	Full API integration test — end-to-end flows for auth, product CRUD, sales, file upload

Phase 4: Mobile Application (Week 7 – 8)

The Android mobile app is built with Kotlin and Jetpack Compose, sharing the same backend API as the web frontend.

Week 7: Android Foundation

Day	Task / Deliverable
Day 1	Android Studio project setup — Kotlin, Jetpack Compose, Retrofit, Hilt DI, Gradle config
Day 2	Authentication screens — Login, Register, JWT token storage in DataStore
Day 3	Inventory screen — product list with image display, low stock indicators, category filter
Day 4	Product detail and add/edit screens — form inputs, image upload via camera/gallery (OWNER only)
Day 5	Retrofit API service layer — all endpoints wired, auth interceptor for Bearer token injection

Week 8: Complete Mobile App

Day	Task / Deliverable
Day 1	Sales recording screen — product picker, quantity input, stock check, sale submission
Day 2	Sales history screen — list view with amounts and timestamps
Day 3	Role-based UI enforcement — STAFF view restrictions, 403 handling with user-facing message
Day 4	UI polish and animations — Compose transitions, touch targets, pull-to-refresh
Day 5	Device/emulator testing; APK generation; mobile documentation

Phase 5: Integration & Deployment (Week 9 – 10)

Final phase covers cross-platform testing, security review, and production deployment of all three platforms.

Week 9: Integration Testing

Day	Task / Deliverable
Day 1	End-to-end testing — full flows on web and mobile: register, add product, record sale, check stock
Day 2	Bug fixes and optimization — address issues from Day 1 testing; query performance checks
Day 3	Security review — JWT expiry, RBAC enforcement, BCrypt rounds, SQL injection via JPA audit
Day 4	Performance testing — API response time targets (<2s), concurrent user simulation
Day 5	Documentation updates — finalize SDD, update README, API docs, architecture diagram

Week 10: Deployment

Day	Task / Deliverable
Day 1	Backend deployment — Railway/Heroku, environment variables, PostgreSQL production instance
Day 2	Web app deployment — Vercel/Netlify, environment config pointing to production API
Day 3	Mobile APK — signed release APK generation and distribution
Day 4	Final integration testing against production environment across all platforms
Day 5	Project submission — repository cleanup, docs/ folder complete, README finalized

8.2 Milestones

The following milestones define the key completion checkpoints across the project lifecycle.

ID	Target	Deliverable	Phase
M1	End of Week 2	All design documents and SDD complete	Planning
M2	End of Week 4	Backend API fully functional and tested	Backend
M3	End of Week 6	Web application complete with all features	Web
M4	End of Week 8	Mobile application complete and APK generated	Mobile
M5	End of Week 10	Full system deployed, integrated, and submitted	Deployment

8.3 Critical Path

The critical path represents the sequence of dependent tasks that directly determines the project completion date. Delays in any of these items will delay the overall schedule.

1. JWT Authentication & Security (Week 3) — All other features depend on a working auth layer.
2. Product CRUD with File Upload (Week 4) — Core business entity; sales recording depends on products existing.
3. Sales Recording with Stock Deduction (Week 4) — Transactional logic must be airtight before frontend builds on it.
4. Google OAuth Integration (Week 3) — External dependency with its own setup complexity; must be done early.
5. External API Integration — Exchange Rate (Week 4) — Must be confirmed working before the dashboard feature is built.
6. Web Frontend API Integration (Week 6) — All frontend work relies on a stable, tested backend.
7. Cross-Platform Integration Testing (Week 9) — Final validation gate before deployment.

8.4 Risk Assessment & Mitigation

The following table identifies the primary technical risks for StockEase and the corresponding mitigation strategies.

Risk	Impact	Likelihood	Mitigation Strategy
Google OAuth integration complexity	**High**	Medium	Implement OAuth first in Week 3; test with sandbox credentials before integration
Exchange rate API downtime or quota limits	**Medium**	Low	Cache last known rate in DB; fallback to stored value if API unavailable
File upload size or storage issues	**Low**	Low	Enforce 5MB limit on backend; validate file type and size before saving
Stock deduction race condition on concurrent sales	**High**	Low	Use database-level transactions with pessimistic locking or optimistic versioning
JWT token leakage or expired token handling	**High**	Medium	Implement refresh token rotation; revoke tokens on logout in DB
Email delivery failure (SMTP)	**Medium**	Medium	Queue email jobs; log failures; send asynchronously to avoid blocking sale flow
Mobile API integration lag behind backend	**Medium**	Medium	Lock API contract early in Week 4; use mock server for mobile dev in parallel

General Risk Mitigation Principles:

- Build the simplest working version of each feature first; layer enhancements only after the core is stable.
- Test all third-party integration points (Google OAuth, Exchange Rate API, SMTP) in Week 3 before dependent features are built.
- Maintain a working Git branch at all times; never commit directly to main without a passing build.
- Lock the API contract at the end of Week 4 so mobile development can proceed in parallel without breaking changes.
- Prioritize the MUST HAVE features from the MoSCoW list before working on SHOULD HAVE or COULD HAVE items.